

KATHRYN: A Cycle-Accurate Control Flow and Resource Abstraction HDL Framework: A Case Study of a RISC-V Out-of-Order Superscalar CPU

Tanawin Devaveja*, *Student Member, IEEE*, and Krerk Piromsopa*, *Member, IEEE*

*Department of Computer Engineering
Chulalongkorn University, Thailand
tanawin1701d@outlook.com, Krerk.P@chula.ac.th

Abstract—We present Kathryn, a framework for abstracting hardware design. As demand for specialized hardware accelerators grows, designers face increasing challenges in manually managing system control flow, structural connections, complex resource structures, and parallelism. Kathryn addresses these challenges through three key innovations: the Hybrid Design Flow (HDF), a hybrid RTL/behavioral cycle-accurate control-flow description; Decentralized Update, which enables operation locality by relaxing structural RTL description; and the Hardware Aggregator, an abstraction for managing complex hardware resource structures.

To evaluate Kathryn, we re-implemented a RISC-V out-of-order superscalar CPU based on RIDECORE [1] as a case study. We further present a qualitative comparison with Chisel that illustrates how Kathryn abstracts and assists hardware control flow, operation locality, and hardware resource management. Compared with the original RIDECORE implementation, Kathryn reduces control, data, and overall lines of code (LOC) by 67.85%, 63.55%, and 56.67%, respectively, while achieving 100% cycle-accurate behavior with no significant overhead in hardware resource utilization.

Index Terms—Hardware Description Language, Hardware simulator, Hardware control flow

I. INTRODUCTION

IN the emerging era of artificial intelligence, edge computing, and highly customizable workloads, chip design faces urgent demands to accelerate application-specific workloads [2], making hardware modeling methodologies critical to structuring the resulting architectures. Existing tools span abstraction levels from low-level gate descriptions to high-level algorithmic representations, yet most fundamentally address two core challenges: *hardware control-flow abstraction* and *hardware resource management abstraction*.

Control-flow abstraction represents hardware states, the logic governing transitions between them, and the flow of data and metadata through the model. Because hardware is inherently parallel [3], designs typically comprise multiple finite state machines (FSMs) [4], so the mechanisms that synchronize and coordinate them are also part of this abstraction. Resource management abstraction, in turn, organizes collections of hardware resources and the methods for updating their values. For example, a reservation station in an out-of-order CPU [5] holds multiple rows of instruction metadata; once selected, an entire row is transferred to an execution unit as a

group, and the logic for group selection, inter-module routing, and allocation constitutes a key aspect of this abstraction.

Based on how tools address these abstractions at different levels, we classify existing hardware modeling methodologies into four main categories.

- 1) Traditional Hardware Description Languages (HDL): HDLs such as Verilog [6], SystemVerilog [7], and VHDL [8] require designers to describe the model's control flow as a structural register-transfer-level (RTL) representation, which we detail further in Section II. This yields **fine-grained, cycle-accurate control** [9] but becomes cumbersome at large scale due to extensive **manual management of control flow, interfaces, and timing**.
- 2) Framework-Assisted Approaches (FAA): FAA embeds hardware construction in software languages. For example, Chisel [10] uses Scala's **parametric types, bundles, and inheritance** to enhance productivity while **maintaining cycle-accurate control**. However, control flow description is still expressed as a **structural register-transfer-level (RTL)** representation, as in traditional HDLs.
- 3) Alternative Hardware Description Languages (AHDL): AHDLs **integrate control-flow and resource abstractions**—pipelining, state machines, and parameterization—directly into their syntax. This reduces modeling overhead but often **lacks the flexibility of FAA**, struggling to express designs beyond what the language or vendor anticipated.
- 4) High-Level Synthesis (HLS): HLS describes algorithms in languages such as C/C++ [11] and compiles them into hardware, largely **abstracting designers from explicit control-flow management**. However, it generally **cannot provide the fine-grained, cycle-accurate control** of the previous three approaches.

Among these, HDL, FAA, and AHDL support cycle-accurate modeling, which is essential for control-intensive, highly optimized designs such as out-of-order CPUs. However, HDL and FAA still express control flow as a **structural register-transfer-level (RTL)** representation and centralize resource-update logic, often forcing manual, unnecessary routing of data and control signals. HLS, in contrast, reduces man-

ual effort but sacrifices fine-grained, cycle-accurate control.

In this paper, we bridge the gap between control-flow abstraction and cycle-accurate control while retaining FAA-style programming features, through the Kathryn framework. Our contributions are:

- 1) Kathryn, an **FAA-style** hardware-construction framework embedded in C++.¹
- 2) The **Hybrid Design Flow (HDF)**, a new control-flow abstraction that expresses cycle-accurate behavior through high-level syntax—eliminating manual control and synchronization logic—while still permitting explicit structural RTL where fine-grained control is needed, so that most designs expressible in HDL or FAA remain fully expressible in Kathryn.
- 3) **Decentralized Update**, which lets designers update a hardware resource’s value at its logical point of origin, avoiding unnecessary control- and data-signal routing.
- 4) The **Hardware Aggregator**, realized by two mechanisms—**Table** and **Slot**—which aggregate multiple hardware components into a bundle and provide facilities to manage its logic.
- 5) A case study implementing a RISC-V out-of-order superscalar CPU, showing that Kathryn measurably reduces not only the control-flow and synchronization logic that designers must write by hand, but also the data logic and the overall code size of the design.

A dedicated discussion section clarifies how Kathryn differs from SystemC-based modeling [12].

The remainder of this paper is organized as follows. Section II reviews related frameworks, languages, and tools; Section III overviews Kathryn and its design advantages; Section IV details the methodology behind its four contributions; Section V evaluates Kathryn on an out-of-order RISC-V use case; and Sections VI and VII discuss the relationship to SystemC and conclude.

II. LITERATURE REVIEW

This section reviews prior work in hardware modeling, focusing on control-flow specification and hardware resource update mechanisms, and highlights remaining limitations.

A. Hardware Control Flow and Hardware Resource Update

In traditional HDLs and most framework-assisted approaches (FAA), designers must explicitly manage hardware control flow, whereas high-level synthesis (HLS) abstracts much of this effort. Each approach involves distinct trade-offs.

In HDL and most FAA approaches (Fig. 1a), the design must be expressed as **structural RTL**: every component and all control logic (state machines, their transitions, and synchronization) is declared and wired by hand. Because structural RTL mirrors physical hardware, each register’s data input can be driven by only one net, and the only way to arbitrate among its writers is combinational selection (`if/else` and `case` in Verilog), which can pick the driving value but cannot express any richer operation. Updates are thus inherently centralized:

```

1 module stage2(
2   input [7:0] curData1,
3   input [7:0] curData2,
4   input curIsShift,
5   input nextReady,
6   output nextAccept,
7   input reset,
8   input clk
9 );
10 reg[1:0] sqrtState1;
11 reg[1:0] sqrtState2;
12 always(posedge clk)
13 begin
14   /// state transition
15   /// is omitted
16 end
17 /// sync and logic
18 /// is omitted
19 endmodule

```

(a) Verilog

```

1 int stage2(
2   uint8_t curData1,
3   uint8_t curData2){
4
5   #pragma HLS PIPELINE
6   int r1=sqrt(curData1)
7   int r2=sqrt(curData2)
8   int rf=r1 * r2;
9   return rf;
10
11 }

```

(b) HLS

Fig. 1: Example of a 2-stage pipelined multi-cycle execution

every writer must funnel its control and data signals into this single driver, even when the operation logically originates elsewhere.

HLS-based designs (Fig. 1b) [13]–[15], by contrast, let designers focus on the algorithm, inferring hardware behavior automatically from minimal directives such as pragmas.

Thus HDL and FAA provide precise, cycle-accurate control at the cost of extensive manual control-flow specification, while HLS abstracts control logic—well suited to dataflow designs—but **sacrifices the cycle-accurate control essential for control-intensive architectures such as out-of-order superscalar processors**.

B. Benefits of Framework-Assisted Approaches

Framework-Assisted Approaches (FAA) construct hardware models using frameworks embedded in host programming languages. By separating hardware semantics from host syntax, FAA lets designers exploit software-engineering features such as object-oriented programming [16], templates, and functional reuse.

Chisel [10], embedded in Scala [17], exemplifies this. In Fig. 2, a base class such as `SizeTracker` (lines 1–5) encapsulates reusable logic and, when inherited by `Bank` (lines 6–12), introduces shared hardware (e.g., the `count` register); parameterizing it by `maxSize` enables scalable generation.

Chisel also uses the host language as a hardware generator: the Scala conditional in lines 15–17 customizes the generated hardware from design parameters without itself being synthesized, letting one codebase describe multiple hardware variants—an advantage typically unavailable in alternative HDLs (AHDL).

C. Current Gaps and How Kathryn Addresses Them

Many framework-assisted approaches have been proposed—Chisel [10], PyRTL [18], PyMTL3 [19], MyHDL [20], and ShakeFlow [21]—embedded in host languages such as Scala, Python [22], and Rust [23], and all support cycle-accurate modeling. They differ in abstraction and philosophy; for example, Chisel offers hardware bundling and parameterized generators, and ShakeFlow abstracts latency-insensitive

¹<https://www.kathryn-tools.org/>

```

1 abstract class SizeTracker
2 (val maxSize: Int) extends Module{
3   val count = RegInit(0.U(width.W))
4   def inc() = count := count + 1
5 }
6 class Bank(rowWidth: Int, wordSize: Int)
7 extends SizeTracker(rowWidth){
8   val io = IO(new Bundle()){
9     val push = Input(UInt(1.W))
10  }
11  when(io.push.asBool){ inc() }
12 }
13 class nWayBank(bankWidth: Int,
14 replacePol: Int) extends Module{
15   if (replacePol == OLDEST_FIRST){
16     /* generator-time logic */
17   }
18   /* instantiate Bank array ... */
19 }

```

Fig. 2: Example of Chisel in FAA

interfaces [24]. Yet in most of these frameworks, control flow, parallelism, and resource management remain **structural RTL**: components and their control logic are declared and wired by hand, with every update centralized to a single driver, as in traditional HDLs.

Cement [25], a Rust-based framework, targets this limitation with an event-based extension—an Event Layer and a control sub-language. This improves control-flow expressiveness but remains insufficient for control-intensive architectures, such as loop-back control flow and hazard-aware pipelining in out-of-order (O3) CPUs [5]. Cement also does not address advanced resource management, such as priority-based selection for grouped registers (e.g., reservation stations) or decentralized updates.

Kathryn builds on the FAA paradigm and shares some syntax with Cement, but adds richer control-flow abstractions with explicit support for control-heavy architectures such as out-of-order CPUs, and addresses the above limitations through Hybrid Design Flow, decentralized update mechanism and Hardware aggregation feature.

In summary, Table I compares existing tools across several dimensions. **Model Type** is the underlying modeling paradigm. **Control Flow** is the highest-level abstraction a tool provides for expressing control flow—the hardware states, their transitions, and the flow of data and metadata defined in Section I—ranging from structural RTL to higher-level constructs. **Cycle-Accurate** denotes support for precise, per-cycle behavior. **Hardware Resource Management** captures the ability to bundle and manage grouped resources, including dynamic selection and control. **Language Assist** reflects how far host-language features are leveraged. **Decentralized Update** indicates whether updates to a single hardware resource can be distributed across sources while preserving designer-controlled ordering and priority.

III. KATHRYN OVERVIEW

This section introduces Kathryn’s features in three parts: (1) **General FAA Features** (module definition, resource types, and logic operations); (2) the **Hybrid Design Flow (HDF) and Decentralized Update**, Kathryn’s core control-flow abstraction for cycle-accurate modeling; and (3) the **Hardware**

Aggregator—the **Table and Slot** abstraction—for managing complex hardware resources.

A. General FAA Features

General FAA features form the foundation of modeling in Kathryn: modules that encapsulate design logic, hardware resource types, and logic operations.

```

1 struct MyModule: Module{
2
3   MyModule(int arg){};
4
5   void flow() override{
6     /* your behavior start here */
7   }
8 }
9
10 mMod(<name>, <MyModule>, args...)

```

Fig. 3: Kathryn Module Declaration

A **Module** (Fig. 3) is a C++ [11] `struct` or `class` inheriting from `Module`; it provides a constructor and implements `void flow()` override, which describes the module’s control flow (next section). Lines 1–8 declare the module and line 10 instantiates it.

Kathryn provides five hardware resource types (Table II). Most mirror their Verilog [6] counterparts, with two relaxations: a **Wire** may be assigned within any procedural block, and a **Nest** groups several resources into a composite that inherits their update constraints.

Kathryn also provides the usual bitwise, shift, comparison, signed-comparison, arithmetic, and bit-extension operators, applicable to all resource types.²

Type	Declare Command	Input Value Trigger
Reg	<code>mReg(#name, #bs)</code>	Pos/Neg Edge Clock
Wire	<code>mWire(#name, #bs)</code>	All Source Change
Nest	<code>g(#sg0, #sg1, ...)</code>	Depend on Source
Mem	<code>mMem(#name, #amtRow, #bs)</code>	Pos/Neg Edge Clock
Value	<code>mVal(#name, #bs, #value)</code>	-

TABLE II: Variable Declaration Types and Update Triggers

B. Hybrid Design Flow (HDF) and Decentralized Update

The HDF describes and controls execution flow in Kathryn. This subsection introduces (1) the **Cycle-Considered Operation (CCO)**, the starting point of control flow; (2) the **Hybrid Design Block (HDB)**, which defines the control-flow syntax; (3) the **Verilog-like** fallback for conventional descriptions; and (4) **decentralized update** commands.

The **Cycle-Considered Operation (CCO)**, shown in Table III, is the assignment operator used to update every hardware resource type. Its source may be either a C++ [11] integer or any readable Kathryn signal. Importantly, Kathryn uses the CCO, as influenced by the enclosing HDB, to build the cycle-accurate control flow.

²The full operator reference is available on the Kathryn website: <https://www.kathryn-tools.org/cppbook/reference/operators/>.

TABLE I: Specifications of tools across hardware design methodologies (field descriptions are provided in Section II-C).

Tools	Model Type	Control Flow	Cycle-Accurate	HW Resource Mgmt.	Lang Assist	Decentralized Update
Verilog	HDL	Structural RTL	Yes	Manual	No	No
Spade	AHDL	emphasized on pipeline abstracted syntax	Yes	Manual	No	No
Silice	AHDL	abstracted via step and pipeline operator	Yes	Manual	No	No
Chisel	FAA	Structural RTL	Yes	Bundle & Vec	Yes	No
PyMTL3	FAA	Structural RTL	Yes	Manual via Host Lang	Yes	No
PyRTL	FAA	Structural RTL	Yes	Manual via Host Lang	Yes	No
MyHDL	FAA	Structural RTL	Yes	Manual via Host Lang	Yes	No
ShakeFlow	FAA	Structural RTL	Yes	Manual via Host Lang	Yes	No
Cement	FAA	Ctrl Sub Lang	Yes	Manual via Host Lang	Yes	No
HLS	HLS	Algorithm with Compiler Directive	No	abstracted through host language	-	-
Kathryn	FAA	Advance Hybrid Design Block	Yes	Table and Slot	Yes	Yes

TABLE III: Assignment Types and Semantics

Type	command	Sensitivity list	Supported Hw Res
Edge Asm	A <=< B	sensitive to next pos/neg edge	Register, MemBlk
Level Asm	A = B	sensitive to state and all source	Register, Wire

The **Hybrid Design Block (HDB)** is Kathryn’s core control-flow abstraction, providing a **hybrid RTL/behavioral, cycle-accurate** description of control flow. Each HDB construct (Table IV) orchestrates CCOs and nested HDBs according to its semantics. Because CCOs are user-defined operators and HDBs are composed from them, designers specify cycle-accurate control flow at the user level.

The *pip/zync* blocks are Kathryn’s most flexible primitives, coupled through a C++ *SyncMeta* object. Though superficially like a valid-ready handshake, they are far more sophisticated: a *pip* orchestrates multiple multi-cycle execution flows and synchronizes with its *zync* automatically, in the exact cycle its completion condition is met. Via *SyncMeta* they add **hazard-aware pipelining** [5]: *holdMaster()/holdSlave()* stall the block in every cycle their call executes, *killSlave()* kills in-flight operations, and stalls propagate backpressure upstream (shown in the evaluation).

Kathryn also provides interrupts: *intrReset(cond)* terminates a block’s state in the same cycle and *intrStart(cond)* reactivates it the next cycle, where *cond* is any readable Kathryn signal.

Figure 4 shows a **cycle-accurate** two-stage pipeline for the example in Fig. 1. The *pipStream* wrapper (line 15) orchestrates the stages: the first (line 16) decodes the instruction to obtain the operands, and the second (lines 17–23) runs two square-root units in parallel (lines 19–21) before multiplying

```

1 void sqrtInt(Operable& x, Reg& y){
2   int bs = x.getOperableSlice().getSize();
3   mReg(xc, bs);
4   seq{
5     par{ xc <= x; y <= x; }
6     Operable& yNext = (y + xc/y) >> 1;
7     cdowhile(yNext < y){
8       y <= yNext;
9     }
10  }
11 }
12
13 void pipstream(Reg& instr){
14   mReg(result, 32);
15   pipStream{
16     auto& [d1, d2] = decode(instr);
17     seq{
18       mReg(r1, 32); mReg(r2, 32);
19       par{
20         sqrtInt(d1, r1); sqrtInt(d2, r2);
21       }
22       result <= r1 * r2;
23     }
24   }
25 }

```

Fig. 4: Kathryn multi-cycle two stage pipeline example

their results in an extra cycle. Both stages are first-level HDBs within *pipStream*, so they execute synchronously.

Lines 1–11 implement a *minimal* Newton–Raphson square root [26] as an HDB, whose latency depends on the input (*curData*)—demonstrating non-deterministic cycle behavior and support for temporary variables and preprocessing. The = on line 6 is not a CCO, as it does not update a hardware resource.

While the HDBs in Table IV express a broad range of control-flow behavior, they may not cover every behavior expressible in Verilog. Designers can therefore revert to an explicit **structural RTL** (Verilog-style) description, subject to three constraints: (1) a resource is sensitive to a single clock edge (e.g., registers, memory) or to all dependent sources (e.g., wires); (2) clock-edge-sensitive elements use non-blocking semantics; and (3) advanced constructs—multiple clock do-

Syntax	seq	par	[c/s/cdo]while	[c/s]if	pick
example	<pre>seq{ b <= a; par{ c1 <= b+1; c2 <= b*2; } }</pre>	<pre>par{ //t1 cdownwhile(a<b) a <= a+1; //t2 cdownwhile(c<d) c <= c+1; }</pre>	<pre>cdowhile(a<b){ a <= a+1; c <= d; }</pre>	<pre>cif(a<b){ a <= a+1; b <= b+1; } celif(a==b){ a <= a+2; } celse{ b <= b+4; }</pre>	<pre>pick{ pif(a<b){ a <= a+1; b <= b+1; } pif(c<d){ c <= c+1; } pdef }</pre>
time constraint	All CCOs and sub-HDBs execute sequentially.	All CCOs and sub-HDBs execute in parallel.	All CCOs and sub-HDBs execute in the manner of the *while master block.	CCOs and sub-HDBs in the matched conditional block execute in the manner of the *if master block.	All matched pif blocks execute in the manner of the pick master block; the block does not wait for all branches to complete.
CCO sub-block	yes	yes	yes	yes	yes (only in pif block)
HDB sub-block	yes	yes	yes	yes	yes (only in pif block)
intr support	yes	yes	yes	yes	yes
ex cycle used	2	max(t1, t2)	[c/cdo] Seq: 2 * LC [c/cdo] Par: 1 * LC [s] Seq: 3 * LC [s] Par: 2 * LC	Seq: 2 if a < b, else 1 Par: 1	Seq: 2 if a < b, else 1 Par: 1
Syntax	zif	ztate	zync[c]	pip	pipStream
example	<pre>zif(a<b){ a <= a+1; b <= b*2; } zelif(a=b){ a <= a+48; } zelse{ b <= a+24; }</pre>	<pre>ztate(a){ zcase(1) {b<=1;} zcase(2) {b<=2;} zcase(3) {b<=3;} zcasedef {b<=0;} }</pre>	<pre>zync(syncVar){ x <= 10; } zynccc(syncVar, cond){ x <= 10; }</pre>	<pre>pip(syncVar){ seq{ a <= 1; b <= 2; } }</pre>	<pre>pipStream{ b <= a; par{ c <= b+1; d <= b+10; } }</pre>
time constraint	All CCOs and sub-HDBs in the matched conditional block execute in a single cycle.	All CCOs and sub-HDBs in the matched constant-case block execute in a single cycle.	Execution starts when the destination pip block with matching meta is free or has just completed execution.	Execution starts when the associated pipeMeta variable is triggered by a matching zync block.	Each first-level CCO or sub-block functions as a pipeline stage, and consecutive blocks execute synchronously.
CCO sub-block	yes	yes	yes	yes	yes
HDB sub-block	only blocks with z prefix (except zync)	only blocks with z prefix (except zync)	only blocks with z prefix (except zync)	yes	yes
intr support	no	no	no	yes	yes
ex cycle used	1	1	1 cycle after pip and cond is ready	1 cycle after zync is ready	infinite
Syntax	s[y/c]Wait	cloop			
example	<pre>syWait(10) scWait(a<b)</pre>	<pre>cloop(idx, 10) a <= a+1</pre>			
time constraint	syWait stalls for a fixed number of input cycles; scWait stalls until its condition is met.	All CCOs and sub-HDBs execute repeatedly for the number of cycles specified in the loop block.			
CCO sub-block	yes	yes			
HDB sub-block	yes	yes			
intr support	yes	yes			
ex cycle used	2	10			

TABLE IV: Kathryn's Hybrid Design Blocks

mains, latches, and tri-state ports—are unsupported. These restrictions are intentional, favoring modeling simplicity and simulation performance.

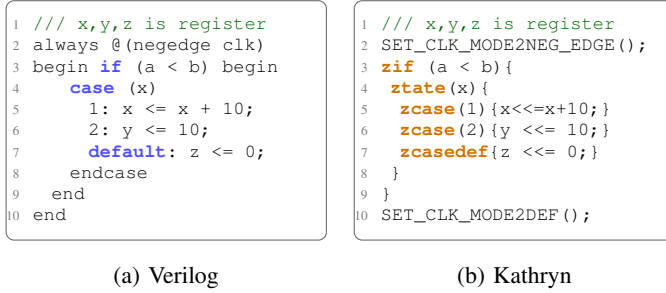


Fig. 5: Kathryn's Verilog style description

This structural-RTL fallback uses *z*-prefixed blocks (*zif*, *zelif*, *zelse*, *ztate*, *zcase*; excluding *zync*), which can also integrate with Kathryn's other HDBs, letting structural RTL and higher-level control flow coexist in a single model. Figure 5 shows functionally identical Verilog (left) and Kathryn (right) implementations that use the same hardware resources.

Kathryn defaults to a positive-edge clock; negative-edge behavior is set explicitly (Fig. 5b, lines 2 and 10).

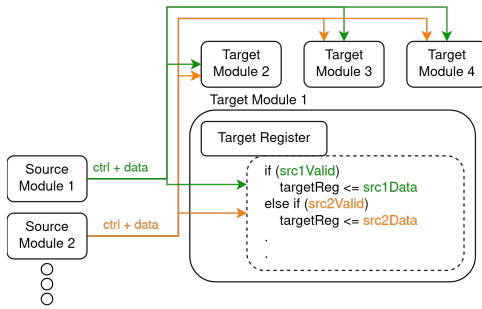


Fig. 6: Traditional Update Procedure

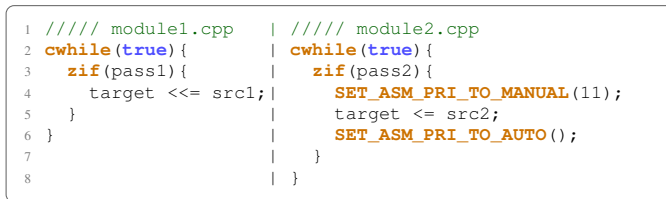


Fig. 7: Kathryn Relax Model

In traditional HDLs and most FAAs (Fig. 6), updating a shared resource requires designers to manually route control and data signals to a centralized update point, forcing explicit centralized logic even when such coordination is unnecessary.

Kathryn instead supports the **decentralized update feature**, which lets designers update a hardware resource **directly from the module that issues the update**, rather than routing it to the module that owns the resource—avoiding unnecessary control- and data-signal routing. Shared resources can thus be updated locally across modules, such as Module 1 and Module 2 in Fig. 7. When conflicts require explicit resolution (e.g., left of line 4 and right of line 5), designers can set the update priority via *SET_ASM_PRI_TO_MANUAL*(*pri*)

or *SET_ASM_PRI_TO_AUTO*() (default 10). We present further examples in the evaluation section.

C. Hardware Aggregator: the Table and Slot Abstraction

Complex structures—such as reservation stations (irregular updates, precise timing) or store buffers (circular queues with random invalidation)—are hard to model with simple resources or naïve bundles. Kathryn addresses this with the **Hardware Aggregator**, which comprises three abstractions: (1) **Slot Meta**, (2) **Reg/Wire Slot**, and (3) **Table**.

```

1 //Slot Meta
2 SlotMeta meta1{
3   {"valid_1", "srcIdx_1", "data_1"},
4   {1, 5, 32}
5 };
6 vector<string> deleteId = {"data_1"};
7 SlotMeta meta2{ {"opcode"}, {7} };
8 SlotMeta meta3 = meta1 + meta2;
9 SlotMeta meta4 = meta3 - deleteId;
10 //Reg/Wire Meta
11 RegSlot rs {meta4};
12 WireSlot ws {meta3};
13
14 Reg& isValid = rs("valid_1");
15 ws("valid_1") = 1;
16 //table
17 Table table {meta4, 16};
18 // HDB support
19 seq{
20   rs <= ws;
21   table[idx] <= rs;
22   table(8)("valid_1") <= 1;
23   rs <= table[idx + 1];
24 }
25 //table reduction finder
26 auto compare = [] (WireSlot& lhs, opr* lidX,
27   WireSlot& rhs, opr* ridX)
28   -> opr{
29     return lhs("srcIdx_1") < rhs("srcIdx_1");
30   };
31 std::tie(ws, idx) = table.doReducBinIdx(compare);
32 //table circular finder
33 auto[result, idx] =
34   table.findMBO_BIDX(true, startPtr,
35     [&](RegSlot& lhs)->opr{
36       return lhs("valid_1") & (lhs("srcIdx_1")==addr);
37     });

```

Fig. 8: Kathryn hardware aggregation primitives

(1) **Slot Meta** is an identifier—a name (string) and index (integer)—for aggregating related resources; instances can be created, merged, or removed at runtime (Fig. 8, lines 1–9).

(2) **Reg/Wire Slot** is a collection of registers or wires laid out by a Slot Meta. Slots can be partially updated with Kathryn assignments or C++ values (line 15) or copied from another slot (line 20), with best-effort name mapping when identifiers differ. All slot updates are CCOs, compatible with every HDB construct.

(3) **Table** is a collection of **Reg Slots** sharing one Slot Meta, supporting static and dynamic update and query, with abstracted extensions for common access patterns:

- **Binary reduction tree:** a user comparator supplied to *doReducBinIdx* (lines 26–30) reduces the table pairwise and returns the surviving row as a Wire Slot with its index (line 31). Reducing *R* rows of *B* bits takes *R* − 1 muxes in total, each one user supplied comparator and one *B*-bit multiplexer, arranged $\lceil \log_2 R \rceil$ levels deep:

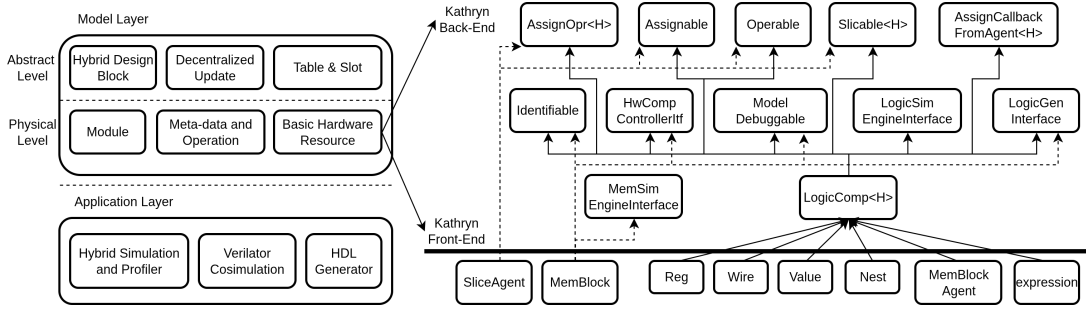


Fig. 9: Kathryn's two-layer architecture (left) and the class hierarchy implementing its fundamental hardware resources (right).

area grows linearly with the table and latency logarithmically.

- **Ordered match search:** `findMBO_BIDX` (lines 33–37) treats the table as a circular queue (via `startPtr`), returning the newest or oldest row matching a predicate (lines 34–36). It reuses the same tree and adds only two bits per row: whether the row matches the predicate, and whether it lies at or after `startPtr`. The tree compares those two bits instead of the row itself, so the circular ordering costs two extra bits per mux.

Both are pure combinational logic and optionally support one-hot index encoding, whose cost grows quadratically with the row count and so suits only small tables. These costs follow from the aggregator's construction rule rather than from synthesis heuristics, so the area and depth of a query are known from the table geometry at design time. Neither call introduces state, and neither adds a cycle: both resolve within the cycle in which they are read.

IV. KATHRYN IMPLEMENTATION

This section describes Kathryn's implementation: its design philosophy, the fundamental hardware components (e.g., registers and wires), update-logic metadata modeling, and the node abstraction underlying the Hybrid Design Flow (HDF).

A. Kathryn Design Philosophy

Kathryn is a **framework-assisted HDL embedded in C++**, comparable to Chisel, rather than a standalone simulator or synthesis tool. It is organized into two layers (Fig. 9 left): the *Model Layer*, the modeling framework that describes synthesizable hardware, and the *Application Layer*, which consumes that model through an **integrated simulator** and an **HDL-generation backend**. **This paper concerns only the Model Layer**; the simulator and HDL generator are implemented as separate backends and lie outside this paper's scope.

The Model Layer represents hardware at two levels. The *physical level* uses fundamental elements (registers, wires, and memory blocks) with signal- and clock-driven update logic, encapsulated in basic modules. The *abstract level* adds higher-level constructs: control-flow abstractions (HDF), complex resources (Table and Slot), and extension-defined features. All abstract constructs are lowered to the physical level, and each layer is kept general enough to extend without modifying the core.

B. Kathryn Fundamental Hardware Resources

The physical level implements fundamental resources (registers, wires, memory blocks) as a C++ [11] class hierarchy (Fig. 9 right), organized into a **Front-End** and a **Back-End**.

The **Front-End** exposes resource objects to designers. Beyond the explicitly declared types of Table II (registers, wires, values, nests, `MemBlock`), it creates helper objects implicitly: **Expression** (operator results), **MemBlockAgent** (a selected memory row, e.g., `myMemBlock[addr]`, promoted from read to write mode on its first write), and **SliceAgent** (static bit selection, e.g., `x(0, 8)` for bits 0 through 7).

The **Back-End** comprises C++ base classes shared by all resources. Of twelve, we describe the eight most essential:

- 1) **Identifiable**: gives each resource and module a unique identifier and localization metadata, letting the simulator and HDL generator declare variables without naming conflicts.
- 2) **Assignable**: holds the update-operation metadata central to update resolution (next section).
- 3) **Operable**: abstracts every resource as a readable signal and implements Kathryn's operators.
- 4) **Slicable / AssignCallbackFromAgent**: enable static bit slicing via a `SliceAgent`; the callback propagates a slice assignment back to the original resource's bit range.
- 5) **HwCompController**: a runtime agent that localizes resources, manages memory ownership, and performs structural integrity checks.
- 6) **LogicSimEngineInterface / LogicGenInterface**: let the simulation engine and the HDL generator, respectively, create an object representing this hardware resource.

C. Kathryn Logic Representation

Kathryn stores the driven logic of each hardware resource as metadata, which also validates its support for Verilog-like [9] descriptions and decentralized updates (Section III-B).

Each updatable resource owns an `Assignable` (Fig. 9 right) that holds its update metadata in a single `UpdatePool` (Fig. 10, line 1), a collection of `UpdateEventBase` instances. `UpdateEventBase` (lines 3–6) is the polymorphic base of a single update event; its `priority` field (line 5) resolves concurrent updates to the same resource (the highest-priority event assigning in a given cycle wins), and its `clkMode` field selects positive-edge, negative-edge, or combinational sensitivity, matching synthesizable Verilog behavior.

```

1 struct UpdatePool { vector<UpdateEventBase*> events; };
2
3 struct UpdateEventBase {
4     // base of all update events
5     UE_TYPE type; bool isLeaf;
6     int priority; ull subPriority; CLOCK_MODE clkMode;
7 };
8 struct UpdateEventBasic : UpdateEventBase {
9     // direct assignment
10    Operable* value; Slice desSlice;
11 };
12 struct UpdateEventCond : UpdateEventBase {
13     // conditional guard on a sub-block
14    vector<Operable*> conditions;
15    vector<UpdateEventBase*> subStmts;
16 };
17 struct UpdateEventGrp : UpdateEventBase {
18     // events in one lexical block
19    vector<UpdateEventBase*> subStmts;
20 };
21 struct UpdateEventSwitch : UpdateEventBase {
22     // multi-branch selector
23    Operable* stateIden;
24    vector<int> subStmtIdxs;
25    vector<UpdateEventBase*> subStmts;
26 };

```

Fig. 10: Driven Logic Structure

```

1 ztate(x){
2     zcase(1){
3         zif(y == 0){ z <= 48; }
4         zif(y == 1){ z <= 24; }
5     }
6 }

```

Fig. 11: Basic Update Logic Example

Kathryn represents driven logic as a tree of `UpdateEventBase` objects collected in an `UpdatePool` (Fig. 10). Four derived types cover the needed constructs: `UpdateEventBasic` (a direct assignment), `UpdateEventCond` (a conditional guard over a sub-block), `UpdateEventGrp` (events sharing one lexical block), and `UpdateEventSwitch` (a multi-branch selector); Fig. 11 shows them in use. Together, these four types cover the block structures of Fig. 5, i.e., the majority of synthesizable Verilog control and update constructs; thus **Kathryn precisely represents Verilog update semantics**. Crucially, the decentralized update mechanism (Fig. 7) **emerges naturally from composing `UpdatePool` and `UpdateEventBase`, with no centralized control logic**.

D. Nodes: The Backbone of Hybrid Design Flow (HDF)

The abstract level of the Model Layer implements the Hybrid Design Flow (HDF) introduced in Section III-B. Its core construct is the *Node*, which represents system state and its associated logic; during lowering, nodes become Fundamental Hardware Resources and Basic Logic Representations by type (Sections IV-B and IV-C).

The Kathryn backend defines nine node types. Three are central:

- 1) **Assignment Node**: a designer-issued assignment (`<=` or `=`) to a resource or aggregator, carrying its target `UpdatePool` and `UpdateEvent`.
- 2) **State Node**: the backbone of most HDBs, representing a single state of a state machine.

- 3) **Synchronize Node**: used by `par` HDBs when sub-block cycle counts are not statically known; it tracks sub-block completion and synchronizes them without adding cycles.

The remaining six are auxiliary: **Pseudo** (a placeholder for cyclic connections), **Dummy** and **Operable** (constant values and readable resources), **Start** (initiates the entry HDB), and **Wait-Condition / Wait-Cycle** (stall until a condition holds or for a fixed number of cycles). The complete mechanism of every node type, including the State Node's formal execution model (its operation and exit status), is documented on the Kathryn website.³ The Start node consumes one cycle after master reset, which Kathryn enforces for implementation convenience; it accounts for the fixed one-cycle offset reported in the evaluation.

V. EVALUATION

This section evaluates Kathryn through two use cases. The primary case study is a RISC-V [27] out-of-order superscalar CPU [5], presented with its evaluation objectives, methodology, CPU overview, and experimental results. A second example, a UART transmit module, illustrates Kathryn's applicability outside the processor domain.

A. RIDECORE Implementation

Evaluation Objectives and Integrity

This evaluation aims to demonstrate that Kathryn can largely replace **structural RTL** for complex designs, and that Kathryn models are **cycle-accurate** against their RTL counterparts, matching internal state on every cycle, with equivalent hardware resource utilization.

To avoid a self-developed baseline, we adopt RIDECORE (RISC-V Dynamic Execution CORE) [1], a Verilog out-of-order RISC-V processor whose microarchitecture is deliberately complex: hazard-aware pipelining, arbitration across multiple data structures, non-deterministic selection logic, and semi-FIFO mechanisms. Handling this complexity implies coverage of designs with simpler control flow. We use RIDECORE with three deliberate changes, none of which alters the intended cycle behavior: its incorrectly implemented gshare predictor is disabled, minor defects are corrected, and its architectural register file is rewritten into a behaviorally identical but non-repetitive form. The rewrite shortens the baseline and therefore works against Kathryn's reported reduction; we quantify it below (*Reduction in Manually Written RTL*). Programs are built with the standard RISC-V GCC toolchain [28], excluding system calls.

RIDECORE Overview

RIDECORE (Fig. 12) is an out-of-order superscalar RISC-V CPU whose six-stage pipeline (fetch, decode, dispatch, wake-up, execute, and complete/retire) pairs a two-wide in-order front end with a multi-lane back end; branch and load/store instructions execute in order, ALU and multiplier instructions

³<https://www.kathryn-tools.org/cppbook/reference/nodes/>

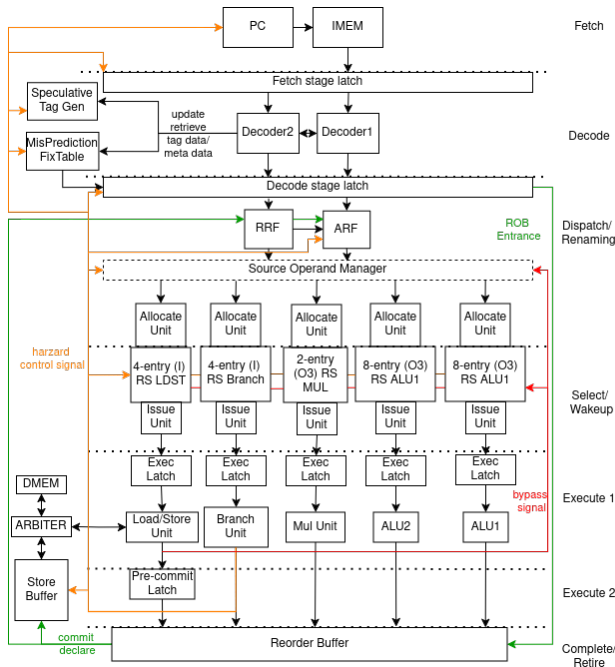


Fig. 12: RIDECORE Design Diagram

out of order. Its 11 major data and metadata structures comprise a Speculative Tag Generator, a Misprediction Fix Table (MPFT), architectural and renamed register files (ARF, RRF), five reservation stations (RSVs; three out-of-order, two in-order), a store buffer, and a reorder buffer. Five speculative tags track in-flight instructions until branch resolution; on misprediction, the back end squashes partially and recovers precisely from six renaming-table copies and the MPFT's recovery metadata. The store buffer, in-order RSV, and reorder buffer are circular FIFOs indexed by a dynamic head pointer, whereas out-of-order RSVs support selective invalidation of arbitrary entries.

Experimental Result

This section reports the experimental results for KRIDECORE and RIDECORE: validation of cycle-accurate behavior in simulation and on FPGA, a lines-of-code comparison, hardware resource utilization after synthesis, and a qualitative comparison with Chisel.

Model Validity: We co-simulate KRIDECORE against the original Verilator-based RIDECORE [1] cycle by cycle, comparing every bit of register state and data on each cycle—treating a register as don't-care only in cycles where it is not in use and cannot influence execution—and validating every memory transaction. Both models run ten workloads (Fibo, Tarai, Cprime, Acker, Hanoi, Matmul, Sort3, Stencil, Stirling, and Komachi) compiled with the standard RISC-V GCC toolchain and a custom linker script, and the two cores are **bit-exactly identical** throughout. We then deploy the model on a KV260 board with PYNQ [29], [30] and rerun the same workloads with cycle tracking enabled. The FPGA measurements match simulation exactly, including cycle counts; Table V reports per-workload cycle usage, where KRIDECORE incurs a fixed one-cycle overhead from the startNode constraint.

TABLE V: KRIDE/RIDE Cycle Usage Across Workloads

Workload	Cycles	Workload	Cycles
Acker	21,758	Matmul	2,707
Cprime	173,018	Sort3	6,488
Fibo	3,352	Stencil	5,192
Hanoi	11,908	Stirling	32,639
Komachi	1,731,861	Tarai	242,444

Simulation time is not a contribution of this paper, but we report it for completeness. Over the 2,231,367 cycles of the ten workloads, RTL simulation takes 140.7 s for KRIDECORE against 8.9 s for RIDECORE, within a 1364.5 s wall clock that also covers Kathryn's other subsystems. KRIDECORE is slower because of Kathryn's just-in-time execution model: the model is compiled externally and reconnects to the simulator at run time, so every signal is simulated, whereas RIDECORE is Verilator-compiled directly into Kathryn. The wall-clock gap stems from a thread-synchronization issue that future work may remove.

Reduction in Manually Written RTL: To quantify manually written RTL objectively, we classify every considered code line into 11 categories using a language-agnostic rule set⁴ applied identically to the Kathryn C++-embedded HDL and the RIDECORE Verilog. The scheme crosses two independent axes. The *domain* axis asks whether a line concerns a **control** (CTRL) or a **data** (DATA) signal, decided by looking up its target (the element it writes, declares, or connects) in a fixed keyword list of semantic roles rather than by human judgment. The *kind* axis asks what the line does: declare a hardware primitive (HWD), compute a value or express control flow (CL), move a value without operators (DT), or connect an interface or port (HC). The two domains and four kinds give eight categories; three domain-less categories cover module or struct declarations (MD), named constants and parameters (PARAM), and host-language scaffolding such as loops, templates, and generate blocks (HLH).

Two handicaps favor RIDECORE. Its 1503 excluded lines (column DC of Table VI) are repetitive code that loops or generate constructs could collapse; we remove them without charging RIDECORE for the loop or generate structure a collapsed version would still need, so the reported 3847-line total is at least as favorable as an actually-collapsed baseline. Its architectural register file, written in a similarly repetitive style, we further rewrote from 949 to 271 behaviorally identical lines, and that version is the one simulated, synthesized and counted throughout. Kathryn's 88 excluded lines are debug instrumentation outside the modeled design. KRIDECORE therefore uses 67.85%, 63.55%, and 56.67% fewer lines for control, data, and overall logic, paying an overhead in PARAM, where named parameters replace string literals for safer Table and Slot field access.

Resource Utilization: We synthesize both KRIDECORE and RIDECORE using Vivado [31] on the Kria KV260 platform with Vivado Implementation Default setting, Table VII reports the post-implementation utilization. LUTRAM (265)

⁴The complete rule set and counting scripts are available at https://github.com/Tanawin1701d/Kathryn/blob/phase-8-tcad-major-revised/src/example/o3/count_measure.

```

1 struct Decode: Module{
2   /// flow control
3   void flow() override{
4     /// pipeline control
5     pip(sync_dc){
6       zyncc(sync_dp, tagGen.isAllGenable()){
7         onDecode(dcd1); /// decode raw instruction to
          register
8         tagMgmt.mpft.onAddNew(tag1, tag2);
9       }
10    }
11  }
12 };

```

```

1 class Decode extends Module{
2   ///pipeline control
3   val busy = RegInit(false.B)
4   io.stall_ID := !io.tagGenable || io.predSuc || io.
      stall_DP
5   io.kill_ID := io.predMis
6   io.busy_ID := busy
7
8   when(busy){
9     if (!io.stall_ID && !io.kill_ID){
10      onDecode(dcd1)
11      busy := io.fetchBusy
12    }.else{ busy := !io.kill_ID }
13    }.else{ busy := io.fetchBusy && !io.kill_ID }
14  }

```

Fig. 13: Decode stage: Kathryn (KRIDECORE, left) versus Chisel (right).

TABLE VI: Per-category LOC breakdown, KRIDECORE (Kathryn) vs. RIDECORE. Rule set and counting scripts⁴.

Category	Kathryn	RIDECORE	Reduction
CTRL_HWD	80.5	292.0	72.43%
CTRL_CL	215.5	572.0	62.33%
CTRL_DT	150.0	285.0	47.37%
CTRL_HC	290.0	1140.0	74.56%
CTRL total	736	2289	67.85%
DATA_HWD	39.0	176.0	77.84%
DATA_CL	122.0	178.5	31.65%
DATA_DT	113.0	273.0	58.61%
DATA_HC	178.0	612.5	70.94%
DATA total	452	1240	63.55%
MD	66.0	121.0	45.45%
PARAM	298.0	172.0	-73.26% [†]
HLH	115.0	25.0	-360.00% [‡]
TOTAL	1667	3847	56.67%
DC (excluded)	88	1503	-

[†]Kathryn uses more lines here: C++ DSL named parameters.[‡]Kathryn uses more lines here: host-language scaffolding.

TABLE VII: Resource utilization comparison.

Resource	KRIDE	RIDE	KRIDE vs. RIDE (%)
LUT	34595	39495	-12.41
LUTRAM	265	265	0.00
Flip-Flop	12435	12536	-0.81
BRAM	6.5	6.5	0.00
DSP	3	9	NON SIG
BUFG	2	3	NON SIG

and BRAM (6.5) are identical and flip-flops differ by only 0.81%, confirming that both designs infer the same memory structures and hold the same architectural state, so the 12.41% LUT reduction reflects combinational logic rather than a smaller machine. KRIDECORE further maps its multiplier onto 3 DSP blocks against RIDECORE's 9 and uses one fewer global buffer, quantities too small to affect overall utilization.

TABLE VIII: Timing comparison at a 20 ns clock constraint.

Metric	KRIDE	RIDE	KRIDE vs. RIDE (%)
Worst data-path delay (ns) [↓]	13.939	14.170	-1.63
Worst negative slack (ns) [↑]	5.600	5.807	-3.56

Percentages are relative to RIDECORE. [↓]lower is better; [↑]higher is better.

Timing and Critical Path: Both designs are constrained to a 20 ns clock and both meet it. Table VIII reports the two metrics, which rank the designs differently because data-

path delay excludes clock skew, uncertainty and endpoint setup: KRIDECORE's worst data-path delay is 1.63% shorter than RIDECORE's, yet its slack is 3.56% smaller. Neither difference is significant, and we claim no timing advantage in either direction. The two worst paths lie in different parts of the machine. RIDECORE's runs through the dispatch stage, which completes rename lookup, register read and operand select in a single cycle. KRIDECORE's runs from the load/store squash logic out to an observation block RAM used for result checking, so its slack is set by that memory rather than by core logic. Both meet the constraint with roughly 5.7 ns of slack, so these figures are lower bounds rather than a measured $F_{\max}$.

Qualitative Explanation: Comparing Kathryn with Chisel and other FAA frameworks is not a matter of counting lines. We make two points. First, Kathryn abstracts the control flow, control logic, and hardware resource management that Chisel still expresses structurally and manually. Second, hardware can be described behaviorally, as a flow of operations rather than a netlist, while keeping two properties HLS abstracts away: **cycle accuracy**, so the per-cycle flow written is what executes, and **decentralized updates**, so each stage updates a shared resource at its logical origin under a designer-specified priority. We do not claim uniform superiority; for many constructs Kathryn is comparable to Chisel, and in some cases more limited. All Chisel counterparts are our own implementations, written with best effort to be idiomatic and compact using Chisel's utility library; shorter formulations may exist but would not change the structural character the comparison shows.

Figs. 13, 15, and 16 compare three KRIDECORE modules, the decode stage, the reservation-station base, and the store buffer, with equivalent Chisel implementations. Decode and dispatch do more than their names suggest: decode receives the instruction from fetch, obtains a speculative tag from the tag generator (which may stall when full), and synchronizes with dispatch; dispatch renames the decoded operands and updates the ARF, RRF, reorder buffer (ROB), and all five RSVs atomically once its handshake with decode, the RRF, and the RSVs succeeds. All of them must also update state on a correct or mispredicted branch. We omit the dispatch listing for space: it mirrors the decode stage in Fig. 13, a `pip/zyncc` block differing only in the transaction it wraps.

In both stages, the `pip/zyncc` constructs (lines 5–6 of Fig. 13) express the entire pipeline control flow, so the

```

1 struct Decode: Module{
2   /// flow control
3   void flow() override{
4     /// pipeline control
5     pip(sync_dc)
6     cdowhile((uopIdx + 1) < uopAmt) seq{
7       par{ /// subDecIdxA, uopAmt, and uopIdx let
8         onDecStageA1 handle it
9         cwhile(subDecIdxA < reqDecIdxA) onDecStageA1(dcd1);
10        cloop(subDecIdxB, 2) onDecStageA2(dcd1)
11      }
12      zyncc(sync_dp, tagGen.isAllGenable()){ /// decode 2
13        cycle
14        onDecodeStage2(dcd1);
15        tagMgmt.mpft.onAddNew(tag1, tag2);
16      }
17    };
18  }
19 }
20
21 class Decode extends Module{
22   ///pipeline control
23   val sIdle :: sParA :: sDc2 :: Nil = Enum(3)
24   val state = RegInit(sIdle)
25   /// last-cycle lookahead, needed to join the two loops
26   val a1End = (subDecIdxA + 1.U) >= reqDecIdxA
27   val a2End = subDecIdxB >= 1.U
28   val uopMore = (uopIdx + 1.U) < uopAmt /// cdowhile
29   condition
30   val dc2Fire = (state === sDc2) && io.tagGenable &&
31     !io.predSuc && !io.stall_DP
32   val accept = (state === sIdle) || (dc2Fire && !uopMore)
33   io.stall_ID := !io.tagGenable || io.predSuc || io.stall_DP
34   io.hold_IF := !accept && !io.predMis /// new: per-instr
35   io.kill_ID := io.predMis
36   io.busy_ID := state === sDc2 || state === sParA
37   io.busy_ID_last := state === sDc2
38   /// state machine
39   when(state === sParA){ /// par: two loops, hand-built join
40     when(subDecIdxA < reqDecIdxA){onDecStageA1(dcd1);}
41     when(subDecIdxB < 2.U){ /// cloop(subDecIdxB, 2) by hand
42       onDecStageA2(dcd1); subDecIdxB := subDecIdxB + 1.U }
43   }
44   when(dc2Fire){
45     onDecodeStage2(dcd1);
46     mpft.onAddNew(tag1, tag2)
47   }
48   when(io.kill_ID){state := sIdle}
49   .elsewhen(dc2Fire && uopMore){ /// cdowhile: next uop
50     subDecIdxB := 0.U;
51     state := sParA
52   }.elsewhen(accept){
53     subDecIdxB := 0.U; state := Mux(io.fetchBusy, sParA,
54       sIdle)
55   }.elsewhen(state === sParA && a1End && a2End){ state :=
56     sDc2 }
57 }

```

Fig. 14: Multi-cycle decode variant, cracking one instruction into several micro-operations: Kathryn (top) versus Chisel (bottom).

designer writes only the stage's transaction: decode translates the raw instruction and registers its speculative tags (lines 7–8), and dispatch, in the same form, renames and writes the reservation station and reorder buffer, admitted by the guard `isdispatable` only when a reservation-station entry, a renaming register, and a renaming-table entry are all available. The `pip` block manages the rest across cycles: readiness, stalls, kills, and back-pressure, both inside the block and with neighboring stages. External events follow the same principle, acting on the affected stage from where they arise rather than through routed control signals. On a correct prediction, `sync_dp.holdMaster()` stalls the stage and propagates back-pressure upstream; on a misprediction, `sync_dc.killSlave()` drops the current block, freeing the stage for an upstream block in the next cycle. Both are

issued from the event source, the branch execution unit's `pip` block, with no signal to route, and Kathryn resolves concurrent requests automatically.

In Chisel, the same semantics are built structurally (Fig. 13, right): explicit per-stage state (the `busy` register, line 3) and dedicated stall, kill, and busy signals routed from every control-event source (fetch, the tag generator, the branch execution unit) into decode and dispatch and folded into their control logic by hand (lines 4–6 and 8–13). Adding a source, or resolving several that fire in the same cycle, means reworking that logic. Utilities such as `Decoupled`, `Queue`, `Arbiter`, and `PriorityEncoder` standardize point-to-point signaling and encapsulate datapath primitives, but when each stage fires and how competing updates are ordered is still assembled by hand. Kathryn instead abstracts the control flow itself, leaving synchronization, signal routing, and priority-preserving updates to the framework.

Fig. 14 extends the decode stage beyond its RIDECORE form. This variant is not implemented in KRIDECORE. We include it because it is realistic in a production CPU: a decoder may take several cycles per micro-operation and may produce several micro-operations from one instruction. It also needs no construct beyond those already introduced.

In Kathryn the stage is still written as the flow it performs: the outer `cdowhile` (line 6) runs once per micro-operation; inside it, `par` (lines 7–10) runs a data-dependent `cwhile` alongside a fixed two-iteration `clloop`; when both finish, the `zyncc` (lines 11–14) sends the micro-operation to dispatch and updates the MPFT, exactly as in Fig. 13. The code reads naturally, top to bottom in the order the hardware executes, and stays cycle-accurate because each HDB fixes the exact cycle of each operation. Chisel builds the same schedule structurally: the decode work itself (`onDecStageA1`, `onDecStageA2`, `onDecodeStage2`) is identical on both sides; every added line exists only to decide in which cycle these calls run. A three-state machine replaces the `busy` register (lines 3–4), the `clloop` counter becomes an explicit guard, increment, and per-iteration reset (lines 20–21, 29, and 32), the `par` join needs last-cycle lookahead (lines 5–7), and fetch and dispatch need different synchronization signals (lines 12–16). The intended flow appears nowhere in the module; the designer must reconstruct it from the state and counter updates.

This difference (Fig. 14) matters most for correctness and maintainability. Control flow may be a small share of a design in many cases, but it is the heart of the design: it decides the cycle in which every datapath element acts, and it works in concert with the Hardware Aggregator and Decentralized Update, so a mistake of a single cycle can grow into a failure of the whole pipeline. Kathryn turns this logic into a readable flow that the framework keeps consistent. In Chisel, even a small change, such as a third iteration for stage A2, forces the designer to rework the join condition, the counters, the state encoding, and both synchronization signals at the same time.

The dispatch stage also shows why decentralized updates and HDBs matter together. A reservation-station entry is written by two independent sources, dispatch, which inserts a new entry, and the bypass network, which forwards

```

1 struct RsvBase{
2   Table _table;
3   /// add entry from dispatch unit
4   void writeEntry(opr& binIdx, WireSlot& iw){
5     SET_ASM_PRI_TO_MANUAL(RSV_WRITE_ENTRY_PRED_PRIORITY);
6     _table[binIdx] <= iw;
7     SET_ASM_PRI_TO_AUTO();
8   }
9   /// update the src register from bypass from exec unit
10  void onBypass(ByPass bp){
11    _table.doCusLogic([&](RegSlot& lhs, int rowIdx){
12      for (int i = 1; i <= 2; i++){
13        zif (lhs(busy) & (~lhs(str(rsValid_) + toS(i))) &
14             (lhs(str(phyIdx_) + toS(i))(0, RRF_SEL) == bp.
15              rrfIdx)){
16          lhs(str(phyIdx_) + toS(i)) <= bp.val;
17          lhs(str(rsValid_) + toS(i)) <= 1;
18        }
19      }
20    });
21  }
22 }

```

```

1 class RsvBase extends Module{
2   val _table = Reg(Vec(entNum, new rsvEntry))
3   val wrEn = io.busy_DP && !io.stall_DP && !io.kill_DP
4   val bpEn = io.bpValid
5
6   /// pool control
7   for (i <- 0 until entNum){
8     for (j <- 0 until 2){
9       when(bpEn && _table(i).busy && !_table(i).rsValid(j)
10          &&
11          (_table(i).phyIdx(j) (RRF_SEL-1, 0) == io.bpRrfIdx
12          )){
13         _table(i).phyIdx(j) := io.bpVal
14         _table(i).rsValid(j) := true.B
15       }
16       when(wrEn && (io.binIdx == i.U)){
17         _table(i) := io.rsvEntry
18       }
19     }
20   }
21 }

```

Fig. 15: Reservation-station base (write & bypass): Kathryn (KRIDECORE, left) versus Chisel (right).

results from every execution unit (nine independent events in the real reservation station). In structural RTL, and in Chisel, both writers must be centralized into the table's single driver, their priority and per-update control managed by hand, typically by ordering when clauses within one module (Fig. 15). Kathryn instead describes each writer where its event occurs: `writeEntry` (dispatch) and `onBypass` (execution units) update the same table independently, and bracketing the dispatch write with `SET_ASM_PRI_TO_MANUAL/SET_ASM_PRI_TO_AUTO` states locally that it precedes the bypass update before restoring automatic resolution. This subsumes the signal routing, source selection, state checking, and interface synchronization Chisel still requires. The RRF, ARF, and ROB are updated the same way, some needing no manual priority at all, so each shared resource is described behaviorally rather than centralizing every writer by hand.

In fairness, Chisel could avoid an explicitly centralized description here by pooling the reservation stations and dispatch logic into one module and separating the writers by an entry's `busy` flag: the bypass network updates an entry whose `busy` bit is set, dispatch writes one whose bit is unset. That shortcut breaks down once a write may overwrite an entry being drained in the same cycle, since `busy` was set earlier and no longer distinguishes the two. The ARF has even more overlapping updates; we omit its listing, which would obscure more than it shows.

The entry write itself shows the Hardware Aggregator's flexibility. Dispatch inserts an entry with one slot assignment, `_table[binIdx] <= iw;` (line 6 of Fig. 15), whose fields are matched to the source bundle by name at model formation time. The sender therefore need not mirror the table's schema: a bundle may supply only some fields or carry extras, and entry fields can be added, removed, or reordered as the design evolves without touching any update site. The same write is shared by five independent reservation stations, each with its own entry layout. This flexibility is a double-edged sword: a misspelled or renamed field may silently go unmatched instead of raising a connection error.

To illustrate the Hardware Aggregator (Table and Slot),

consider store-to-load forwarding: a load checks the store buffer for an earlier in-flight store that can supply its value directly. The buffer is a circular queue, so the load must select the newest valid entry whose address matches, measured from the queue's dynamic head pointer; in Fig. 17 the head is at entry 2, so entry 1 is selected. In Kathryn (Fig. 16), this search is one call to the Table aggregator `findMBO_BIDX` (lines 7–10), parameterized by the head pointer `finPtr` and a predicate selecting busy entries with a matching address; the round-robin priority is realized inside the aggregator by a binary reduction tree over an augmented compare tree, with no explicit rotation. Chisel comes close, expressing the same search as a match vector, a rotation, `PriorityEncoder`, and `Reverse`. Both versions are cycle-accurate and similar in size, so the advantage is small and lies in who manages the procedure: in Kathryn the designer states what to find and the aggregator decides how; in Chisel the rotation, highest-bit selection, and wrap-around rotation back are built by hand, each easy to get subtly wrong, so the Kathryn version simply leaves less room for such mistakes. This search recurs beyond CPUs: although not implemented in this work, an FR-FCFS memory controller picking the oldest request that hits the open DRAM row would reuse `findMBO_BIDX` with a row-hit predicate.

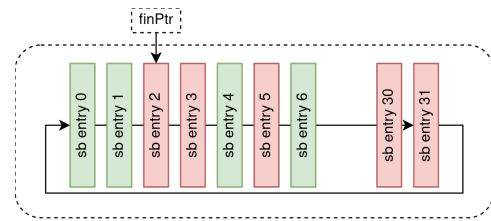


Fig. 17: Store-to-load forwarding in the store buffer.

The cost follows from the table geometry. The store buffer holds 32 entries of 40 bits, and the reduction carries a 5-bit row index alongside the row. Each entry is compared against the load address once and tagged with two extra bits, after which 31 merges each select $40 + 2 + 5 = 47$ bits, giving $31 \times 47 = 1,457$ bits of two-input multiplexing at a depth of 5 levels. Matching dominates, since the circular ordering adds only two bits per merge.


```

1 struct StoreBuf{
2   mReg(finPtr, STBUF_ENT_SEL);
3   Table _table;
4   mMem(daytas , STBUF_ENT_NUM, DATA_LEN);
5   /// find first match round robin.
6   pair<opr&, opr&> searchNewest(opr& addr){
7     auto[result, binIdx] = _table.findMBO_BIDX(true, finPtr,
8       [&](RegSlot& lhs)->opr&{
9         return lhs(busy) & (lhs(memAddr) == addr);
10      });
11     return {result(busy) & (result(memAddr) == addr),
12           daytas[binIdx]};
13   }
14 }

```

```

1 class StoreBuf extends Module{
2   val finPtr = RegInit(0.U(entSel.W))
3   val busy = RegInit(VecInit(Seq.fill(entNum)(false.B)))
4   val memAddr = Reg(Vec(entNum, UInt(addrLen.W)))
5   val daytas = Reg(Vec(entNum, UInt(dataLen.W)))
6   /// find first match RR: rotate, search, rotate back
7   def searchNewest(addr: UInt): (Bool, UInt) = {
8     val hitVec = VecInit((0 until entNum).map(i =>
9       busy(i) && (memAddr(i) === addr))).asUInt
10    val hitRot = hitVec.rotateRight(finPtr)
11    val newest = (entNum-1).U - PriorityEncoder(Reverse(
12      hitRot))
13    val binIdx = newest+finPtr // rotate back, wraps mod N
14    (hitRot.orR, daytas(binIdx))
15  }

```

Fig. 16: Store buffer (newest-match search): Kathryn (KRIDECORE, left) versus Chisel (right).

```

1 /// caller side: hand the frame over through the pip
2 handshake
3 zync(uart.syn){ uart.shiftReg <= g(one, data, zero); } //
4 / stop,data,start
5 /// transmitter side
6 tx = shiftReg(0);
7 pip(syn){
8   seq cloop(bid, 10){
9     syWait(_cyclesPerBit - 1);
10    zif (bid != 9)
11      shiftReg <= g(one, (shiftReg)(1, 10));
12  }
13 }

```

```

1 io.tx := shiftReg(0)
2 switch(state) {
3   is(sIdle) {
4     clockCount := 0.U
5     when(io.in.valid) {
6       shiftReg := Cat(1.U(1.W), io.in.bits, 0.U(1.W))
7       bitCount := 0.U
8       state := sSend
9     }
10  }
11  is(sSend) {
12    when(baudTick) {
13      clockCount := 0.U
14      shiftReg := Cat(1.U(1.W), shiftReg(frameBits - 1, 1))
15      when(bitCount === (frameBits - 1).U){ state := sIdle }
16      .otherwise{ bitCount := bitCount + 1.U }
17    }.otherwise { clockCount := clockCount + 1.U }
18  }
19 }

```

Fig. 18: UART transmitter (8N1 frame): Kathryn (left) versus Chisel (right).

B. UART Transmitter Example

The same control abstraction applies to small control-dominated modules. Our UART transmitter (Fig. 18) serializes a 10-bit 8N1 frame (a low start bit, eight data bits least-significant first, and a high stop bit), holding each bit for `_cyclesPerBit` cycles. In Kathryn the module is the sequence it performs: the `pip` block (line 5) synchronizes with the caller, whose `zync` loads the frame in one statement (line 2), and `seq cloop(bid, 10)` runs one bit period per iteration (lines 6–10), holding the line with `syWait` and shifting ones in so it rests idle-high after the stop bit; no transmit state, bit counter, baud comparator, or handshake logic is written by hand. Chisel builds the same functionality structurally, from a two-state machine (`sIdle`, `sSend`), a `clockCount` register with a `baudTick` comparator, a `bitCount` register, and an explicit `io.in.valid` handshake. Each piece is simple, but the flow of the frame appears nowhere in the code: it is implied by the state and counter updates, so the designer must reconstruct it mentally.

VI. DISCUSSION

Because Kathryn is embedded in C++, its relationship to SystemC [12] deserves comment. SystemC models behavior through threads, events, and dynamic constructs, generally neither cycle-deterministic nor synthesizable by construction. Framework-assisted approaches—Chisel [10], Cement [25], and ShakeFlow [21]—instead separate the modeling layer from the host language, so user-level models are cycle-accurate and synthesizable by design; Kathryn follows that paradigm,

including for Hybrid Design Blocks. Synthesizable SystemC subsets exist but read as structural RTL, and the HLS tools that relax those restrictions (e.g., Stratus [13], Catapult [14]) determine cycle behavior themselves.

A second matter is what Kathryn deliberately does not model: multiple clock domains, latches, and tri-state ports (Section III-B). The three exclusions carry different weight. Multiple clock domains are the real boundary: a design whose top level crosses clock domains cannot be described entirely in Kathryn. In practice, each synchronous block is described in Kathryn and the blocks are then integrated in a conventional flow, with the clock-domain-crossing logic written outside the model. This is a limit of the current implementation rather than of the method, since each resource already carries its own update metadata with an explicit clock mode (Section IV-C), but we claim no support we have not built. The other two exclusions matter less in practice: modern synchronous flows discourage or prohibit latch inference, and tri-state signaling inside the chip has largely been replaced by multiplexers on both FPGA and ASIC targets, surviving mainly at pad rings and off-chip buses. What the three exclusions buy is simplicity: every resource is sensitive to a single clock edge or to its sources, so the update semantics of Section IV-C cover every case and the simulator’s evaluation order is well defined. The cost is that designs depending on these constructs fall outside the modeled subset; the benefit is that everything inside it is cycle-deterministic by construction.

VII. CONCLUSION

This paper presents **Kathryn**, a cycle-accurate and synthesizable hardware modeling framework embedded in C++ [11]. Kathryn introduces three modeling features: **Hybrid Design Flow (HDF)**, which provides explicit cycle-accurate control-flow abstractions; **decentralized updates**, which relax the single-driver constraint of structural RTL; and a **Hardware Aggregator** that facilitates the design of complex hardware resource structures. Using a highly complex out-of-order RISC-V core (KRIDECORE) as a case study, Kathryn reduces control, data, and overall lines of code by 67.85%, 63.55%, and 56.67% relative to RIDECORE, while maintaining nearly identical hardware resource utilization.

Future work includes optimizing the generated HDL, finding more efficient ways to implement Kathryn models in hardware, and studying large-scale designs.

ACKNOWLEDGMENT

Declaration of generative AI use. The authors used Claude (Anthropic) to shorten the text and correct its grammar, to formalize the author-conceived line-classification rule set and apply its tags to the KRIDECORE and RIDECORE sources, and to supplement the authors' own study of Chisel practice, so that the qualitative comparison uses idiomatic Chisel. **Every tag assigned to the KRIDECORE and RIDECORE sources was inspected line by line by the authors, as was every line of text the tool altered.** The authors reviewed and edited all AI-assisted content and take full responsibility for the content of this article.

REFERENCES

- [1] Arch Lab, Tokyo Institute of Technology, "ridecore/ridecore," original-date: 2016-03-29T05:51:08Z. [Online]. Available: <https://github.com/ridecore/ridecore>
- [2] L. Truong and P. Hanrahan, "A golden age of hardware description languages: Applying programming language techniques to improve design productivity," in *3rd Summit on Advances in Programming Languages (SNAPL 2019)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2019.
- [3] J. O'Donnell, "Parallelism through digital circuit design," 01 2007.
- [4] S. L. Harris and D. Harris, "Digital Design and RISC-V Computer Architecture Textbook," pp. 1–5, publisher: IEEE.
- [5] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Fifth Edition: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.
- [6] "IEEE Standard for Verilog Hardware Description Language," *IEEE Std 1364-2005 (Revision of IEEE Std 1364-2001)*, pp. 1–590, 2006.
- [7] "IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language," *IEEE Std 1800-2023 (Revision of IEEE Std 1800-2017)*, pp. 1–1354, 2024.
- [8] "IEEE Standard for VHDL Language Reference Manual," *IEEE Std 1076-2019*, pp. 1–673, 2019.
- [9] S. Palnitkar, "Verilog HDL: a guide to digital design and synthesis," place: Switzerland, EuropePublisher: SunSoft Press.
- [10] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniec, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 1216–1225.
- [11] ISO/IEC, "Programming languages — c++, "Draft International Standard N4660, March 2017.
- [12] "IEEE Standard for Standard SystemC® Language Reference Manual," *IEEE Std 1666-2023 (Revision of IEEE Std 1666-2011)*, pp. 1–618, 2023.
- [13] Stratus High-Level Synthesis | Cadence. [Online]. Available: https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/stratus-high-level-synthesis.html
- [14] Siemens Digital Industries Software, "Catapult High-Level Synthesis." [Online]. Available: <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>
- [15] AMD, "Vitis Embedded Software Development (UG1400)," 2024. [Online]. Available: <https://docs.amd.com/r/en-US/ug1400-vitis-embedded/Installation-Requirements>
- [16] R. Chopra, *Object Oriented Programming Using C++*. NEW AGE International Publishers.
- [17] M. Odersky and al., "An overview of the scala programming language," EPFL Lausanne, Switzerland, Tech. Rep. IC/2004/64, 2004.
- [18] J. Clow, G. Tzimpragos, D. Dangwal, S. Guo, J. McMahan, and T. Sherwood, "A pythonic approach for rapid hardware prototyping and instrumentation," 09 2017, pp. 1–7.
- [19] S. Jiang, P. Pan, Y. Ou, and C. Batten, "PyMTL3: A Python Framework for Open-Source Hardware Modeling, Generation, Simulation, and Verification," *IEEE Micro*, vol. 40, no. 4, pp. 58–66, 2020.
- [20] "myhdl/myhdl," original-date: 2015-02-07T19:50:27Z. [Online]. Available: <https://github.com/myhdl/myhdl>
- [21] S. Han, M. Jang, and J. Kang, "ShakeFlow: Functional Hardware Description with Latency-Insensitive Interface Combinators," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 702–717.
- [22] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.
- [23] N. D. Matsakis and F. S. Klock II, "The rust language," in *ACM SIGAda Ada Letters*, vol. 34, no. 3. ACM, 2014, pp. 103–104.
- [24] L. P. Carloni, "From latency-insensitive design to communication-based system-level design," *Proceedings of the IEEE*, vol. 103, no. 11, pp. 2133–2151, 2015.
- [25] Y. Xiao, Z. Luo, K. Zhou, and Y. Liang, "Cement: Streamlining FPGA Hardware Design with Cycle-Deterministic eHDL and Synthesis," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA '24. Association for Computing Machinery, pp. 211–222.
- [26] B. Ayyub and R. McCuen, *Numerical Methods for Engineers*, ser. Prentice Hall international editions. Prentice Hall.
- [27] A. Waterman, Y. Lee, R. Avizienis, H. Cook, D. Patterson, and K. Asanovic, "The RISC-V instruction set," in *2013 IEEE Hot Chips 25 Symposium (HCS)*, 2013, pp. 1–1.
- [28] RISC-V Options (Using the GNU Compiler Collection (GCC)). [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>
- [29] Kria KV260 Vision AI Starter Kit. [Online]. Available: <https://www.amd.com/en/products/system-on-modules/kria/k260/kv260-vision-starter-kit.html>
- [30] PYNQ. [Online]. Available: <http://www.pynq.io/>
- [31] AMD, "Vivado Design Suite User Guide: Design Methodology (UG949)." [Online]. Available: <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Vivado-Design-Suite-User-and-Reference-Guides>

Tanawin Devaveja received the M.Sc. degree from Chulalongkorn University, Thailand, and is currently an independent researcher. His research interests include computer architecture, hardware design, and hardware–software codesign.



Krerk Piromsopa (Member, IEEE) received the BEng and MEng degrees in computer engineering from Chulalongkorn University, Thailand, in 1998 and 2000, respectively. In 2003, he was awarded a scholarship from the Royal Thai Government for pursuing his PhD degree in computer science, which he received from Michigan State University in 2006. He has been a lecturer and later a professor at Chulalongkorn University since 2001. His research interests are in computer security, computer architecture, distributed systems, and big-data in health-care

applications. He has been a member of the IEEE and the IEEE Computer Society for 20 years.